INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

Lab13: Session Hijack

Lab 13: Session Hijacking and Mitigation

Objective

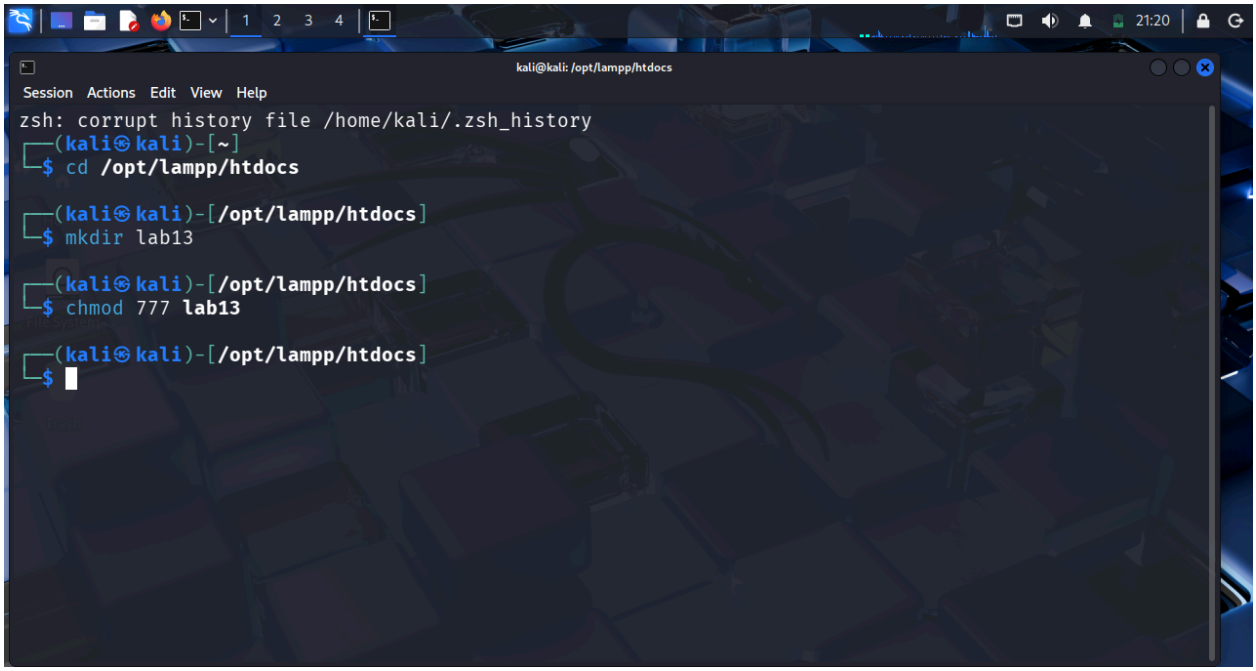
The purpose of this lab was to:

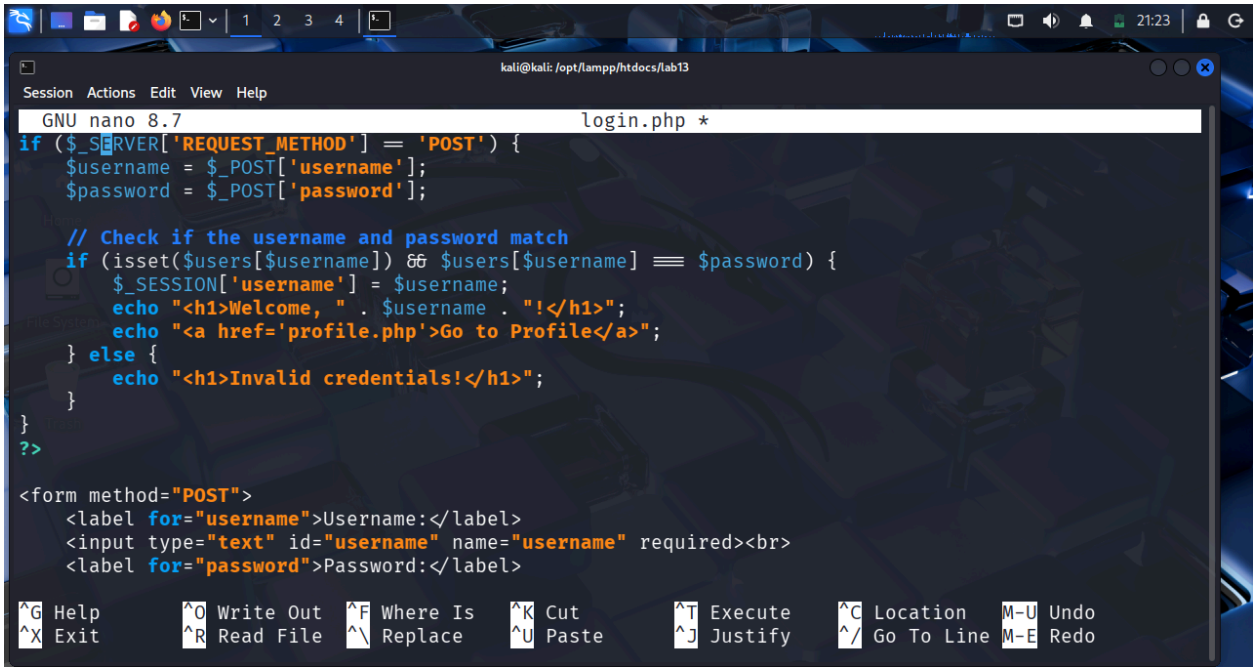
1. Understand how session hijacking works.
2. Simulate a session hijacking attack.
3. Implement mitigation techniques to prevent session hijacking.

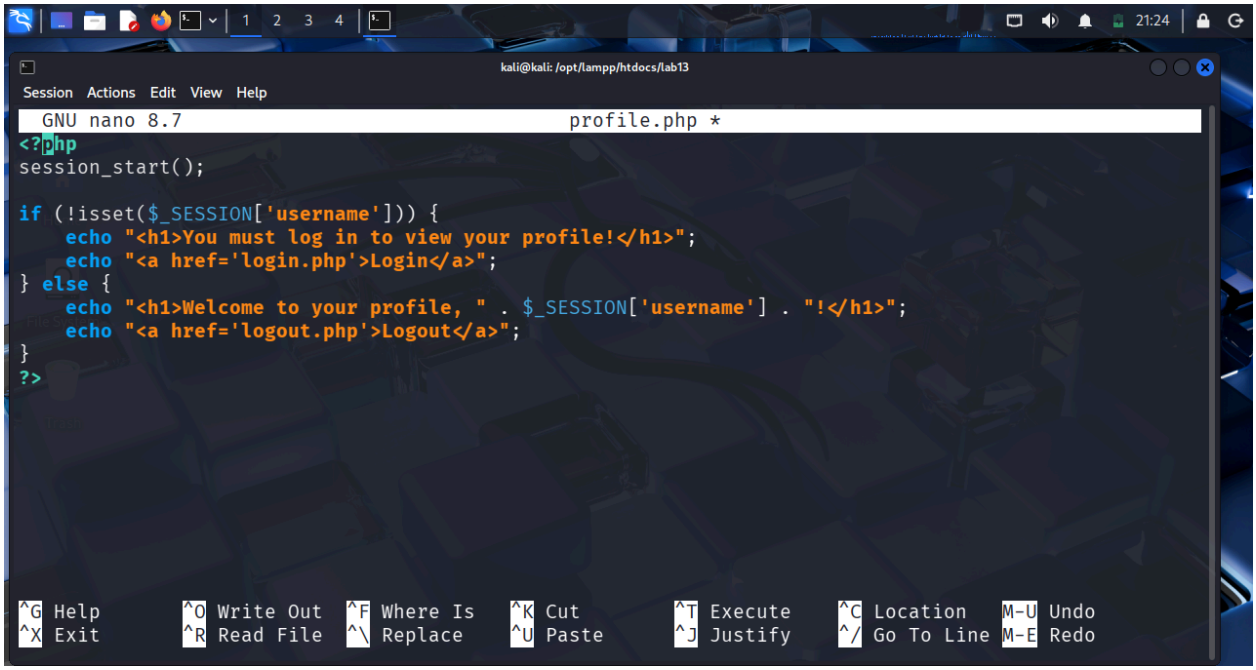
Task 1: Simulating Session Hijacking

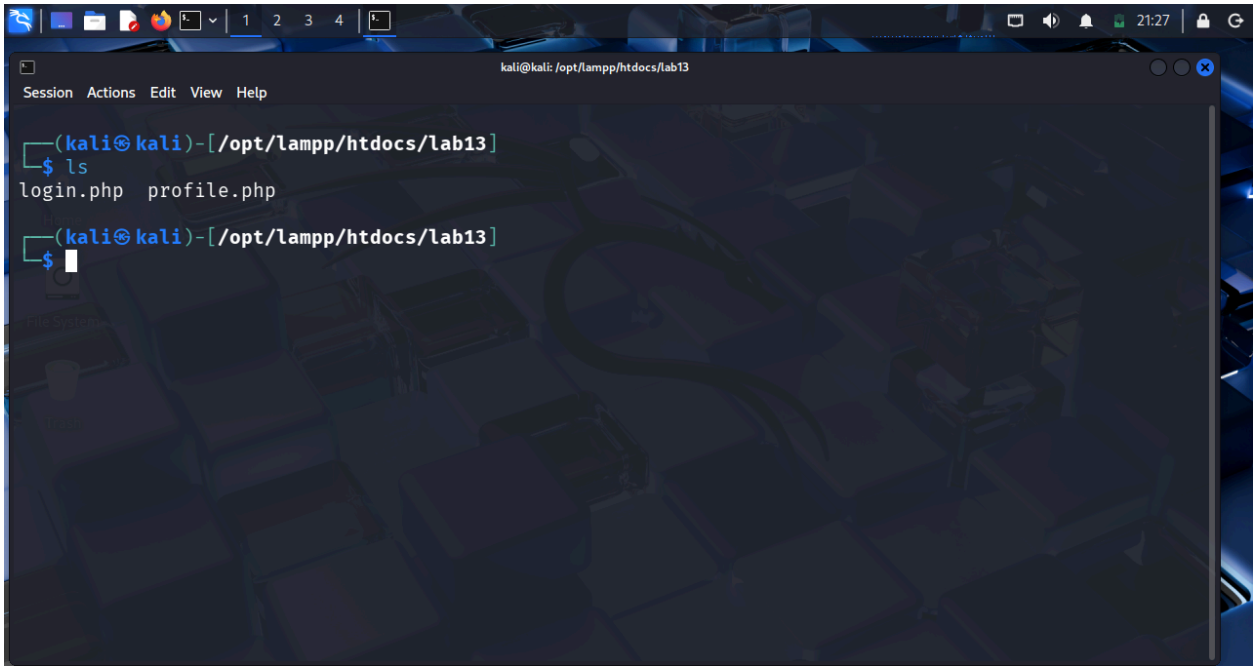
1. Setup of Vulnerable Application

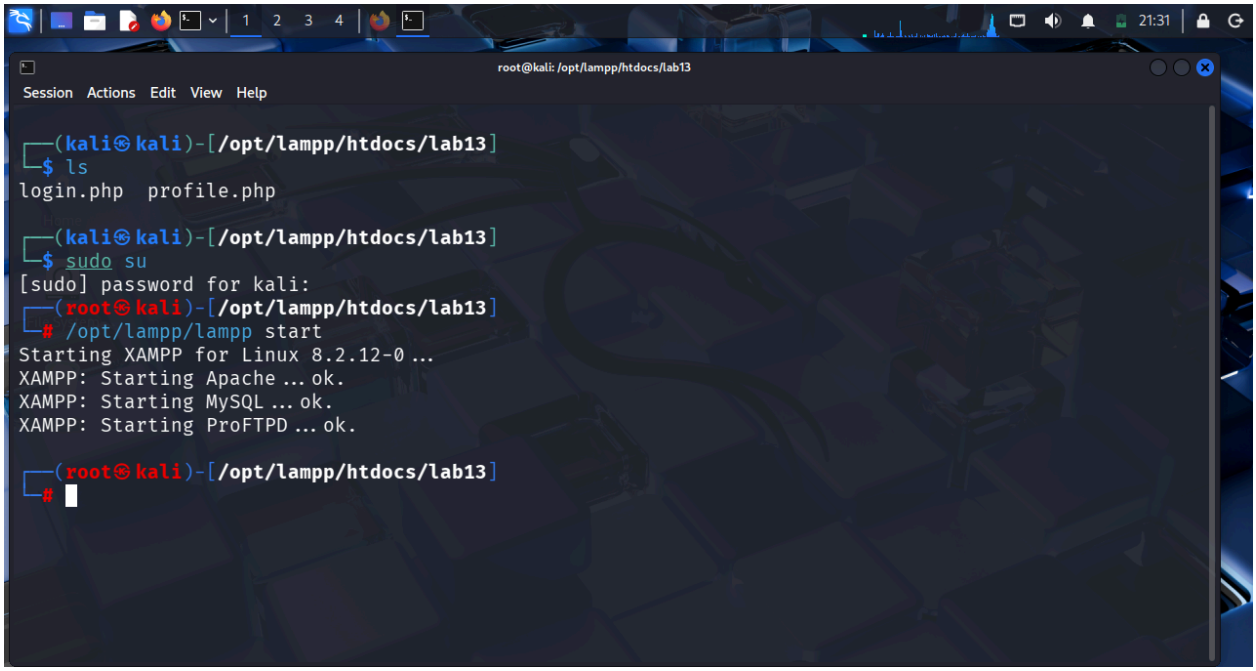
A simple PHP login system was created using login.php and profile.php.
The application used PHP sessions (PHPSESSID) to maintain user authentication.






1 2 3 4
21:24
kali@kali: /opt/lampp/htdocs/lab13
Session Actions Edit View Help
GNU nano 8.7 profile.php *
<?php
session_start();
if (!isset($_SESSION['username'])) {
echo "<h1>You must log in to view your profile!</h1>";
echo "Login";
} else {
echo "<h1>Welcome to your profile, " . $_SESSION['username'] . "!</h1>";
echo "Logout";
}
?>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo





2. Successful Login

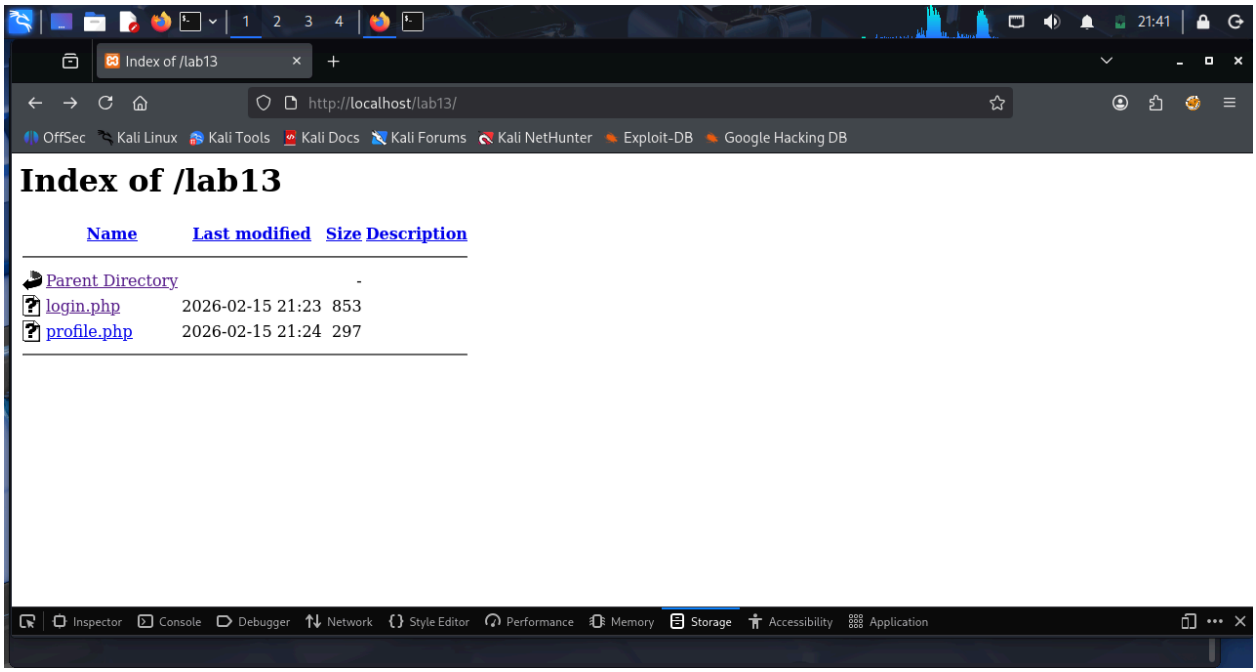
The application was accessed via:

http://localhost/Lab13/login.php

Login credentials used:

- Username: admin
- Password: password123

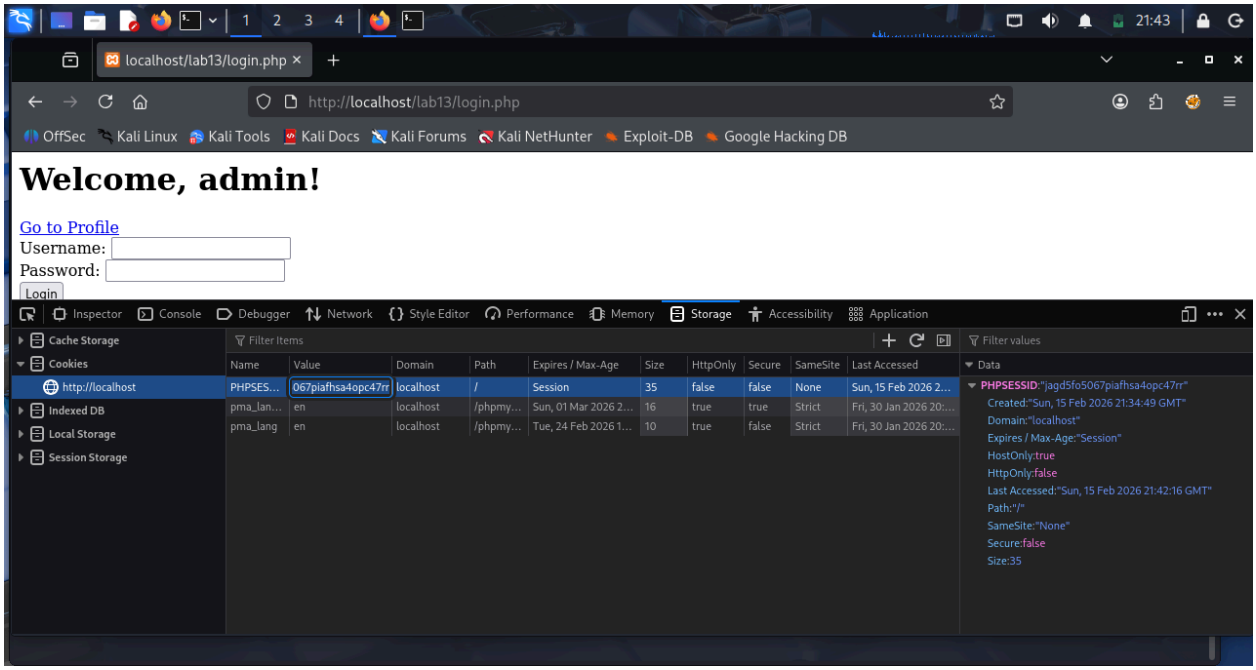
After successful login, the system created a session and granted access to the profile page.



3. Extracting the Session ID

Using browser Developer Tools:

- Navigated to **Application → Cookies → localhost**
- Located the session cookie named PHPSESSID
- Copied the session ID value



4. Simulating the Attack

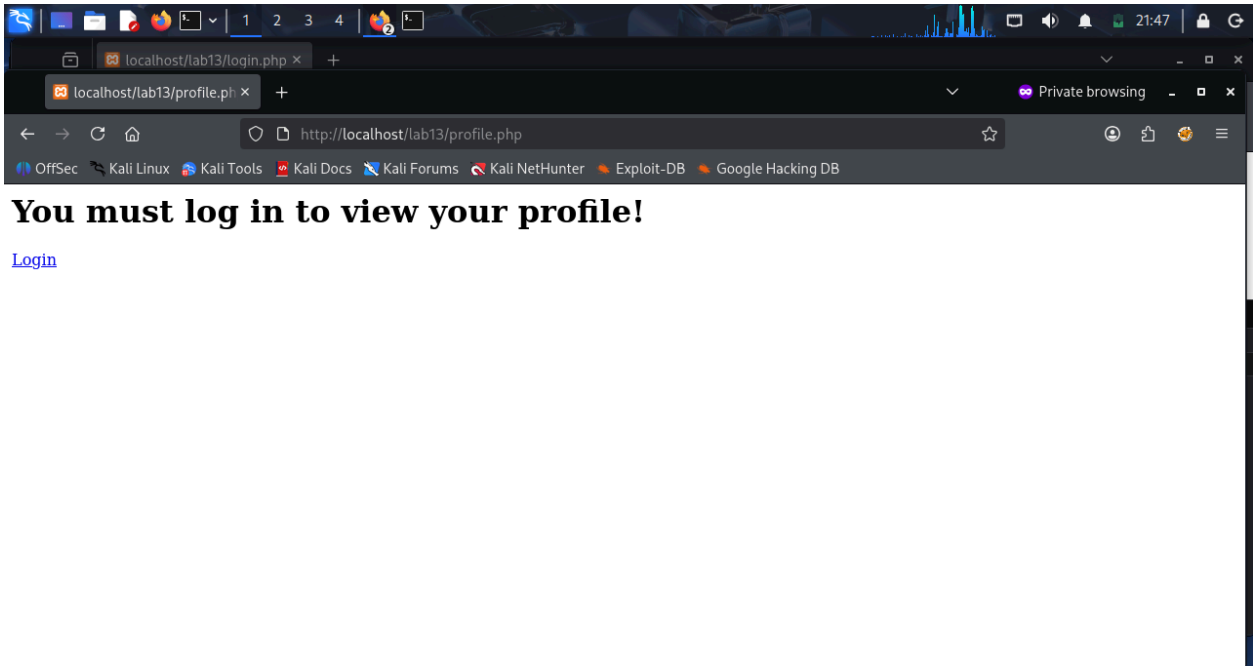
An Incognito window was opened.

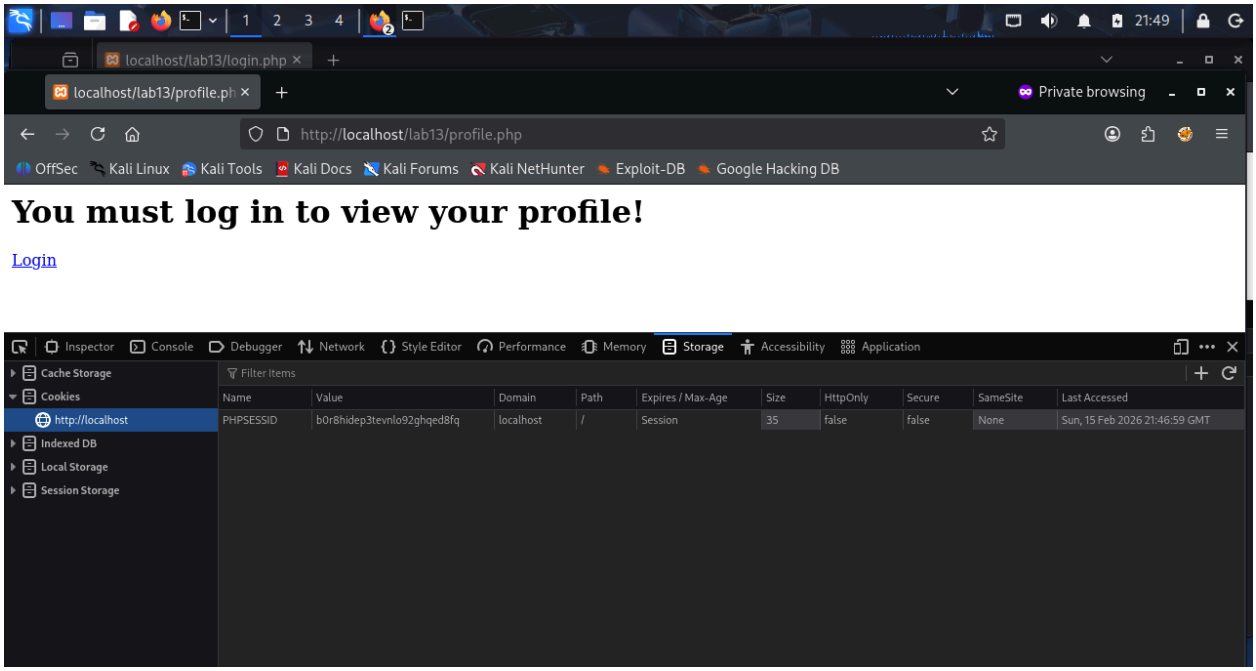
Steps performed:

1. Navigated to profile.php
2. Manually inserted the stolen PHPSESSID into cookies
3. Refreshed the page

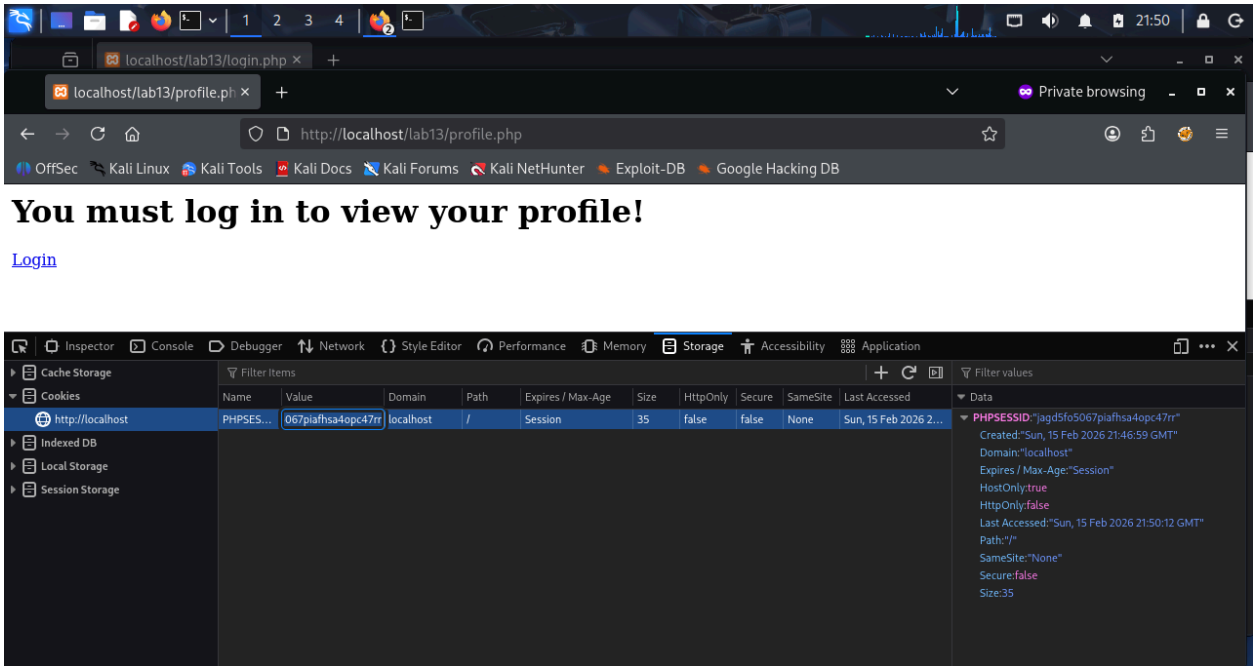
Result:

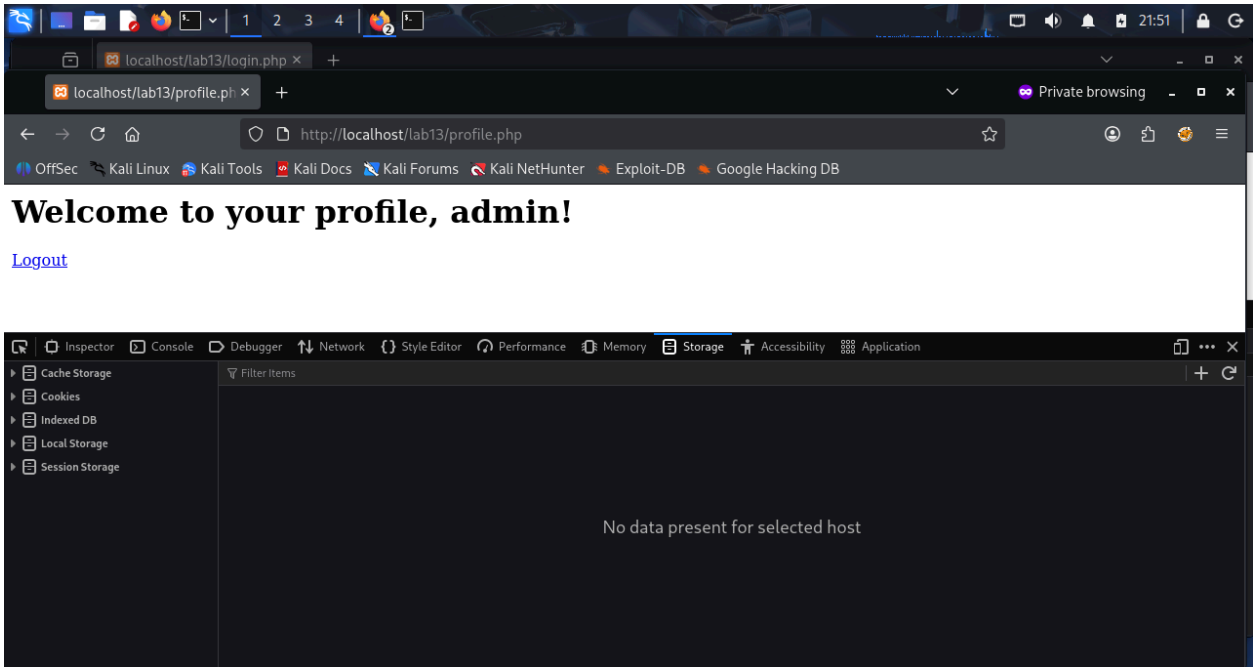
The profile page was accessed without logging in, proving that session hijacking was successful.


21:47
localhost/lab13/login.php
localhost/lab13/profile.ph
Private browsing
http://localhost/lab13/profile.php
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
You must log in to view your profile!
Login


21:49
localhost/lab13/login.php
localhost/lab13/profile.ph
Private browsing
http://localhost/lab13/profile.php
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
You must log in to view your profile!
Login

Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Cache Storage
Cookies
http://localhost
Indexed DB
Local Storage
Session Storage
Filter Items
Name
Value
Domain
Path
Expires / Max-Age
Size
HttpOnly
Secure
SameSite
Last Accessed
PHPSESSID
b0r8hidep3tevnlo92ghqed8fq
localhost
/
Session
35
false
false
None
Sun, 15 Feb 2026 21:46:59 GMT


21:50
localhost/lab13/login.php
localhost/lab13/profile.ph
Private browsing
http://localhost/lab13/profile.php
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
You must log in to view your profile!
Login
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Cache Storage
Cookies
http://localhost
Indexed DB
Local Storage
Session Storage
Filter Items
Filter values
Name
Value
Domain
Path
Expires / Max-Age
Size
HttpOnly
Secure
SameSite
Last Accessed
PHPSES...
067piafhsa4opc47rr
localhost
/
Session
35
false
false
None
Sun, 15 Feb 2026 2...
Data
PHPSESSID:"jagd5fo5067piafhsa4opc47rr"
Created:"Sun, 15 Feb 2026 21:46:59 GMT"
Domain:"localhost"
Expires / Max-Age:"Session"
HostOnly:true
HttpOnly:false
Last Accessed:"Sun, 15 Feb 2026 21:50:12 GMT"
Path:"/"
SameSite:"None"
Secure:false
Size:35


21:51
localhost/lab13/login.php
localhost/lab13/profile.ph
Private browsing
http://localhost/lab13/profile.php
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Welcome to your profile, admin!
Logout
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Cache Storage
Cookies
Indexed DB
Local Storage
Session Storage
Filter Items
No data present for selected host

Explanation of the Vulnerability

Session hijacking works by stealing a valid session ID from a legitimate user.
Since the server uses the session ID to identify authenticated users, an attacker who obtains it can impersonate the victim without knowing their password.

The vulnerability existed because:

- The session ID was not regenerated after login.
- No session timeout was implemented.
- Cookie security flags were not configured.

Task 2: Mitigation Measures Implemented

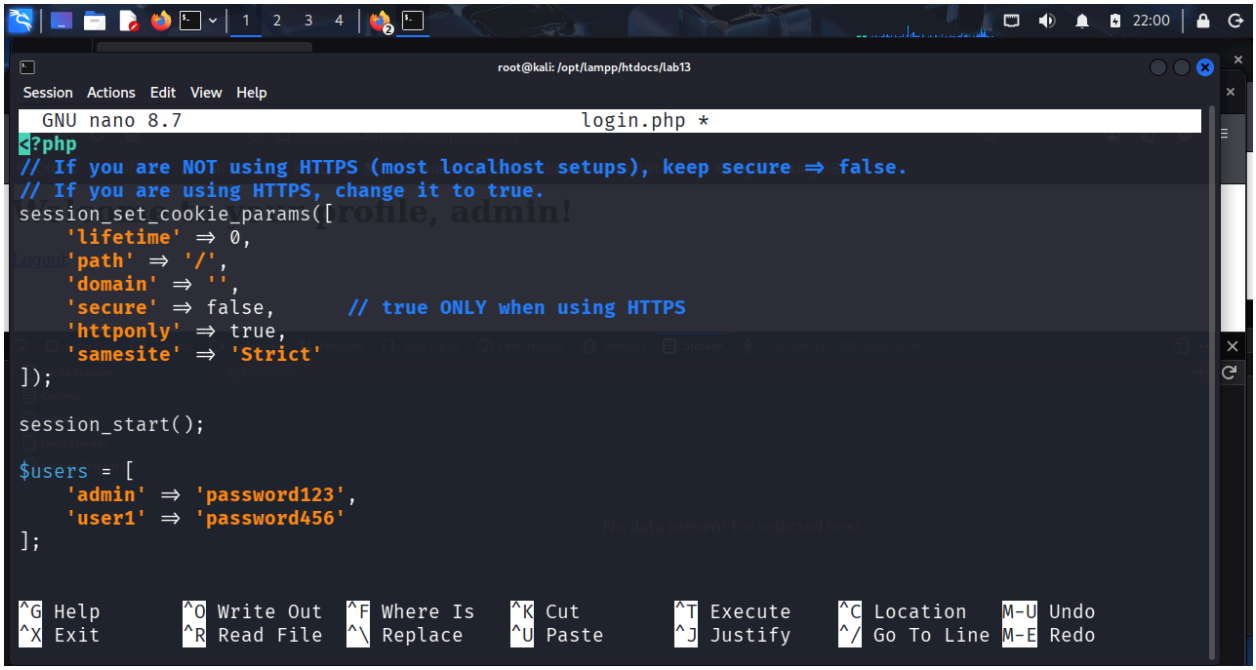
The following security improvements were added:

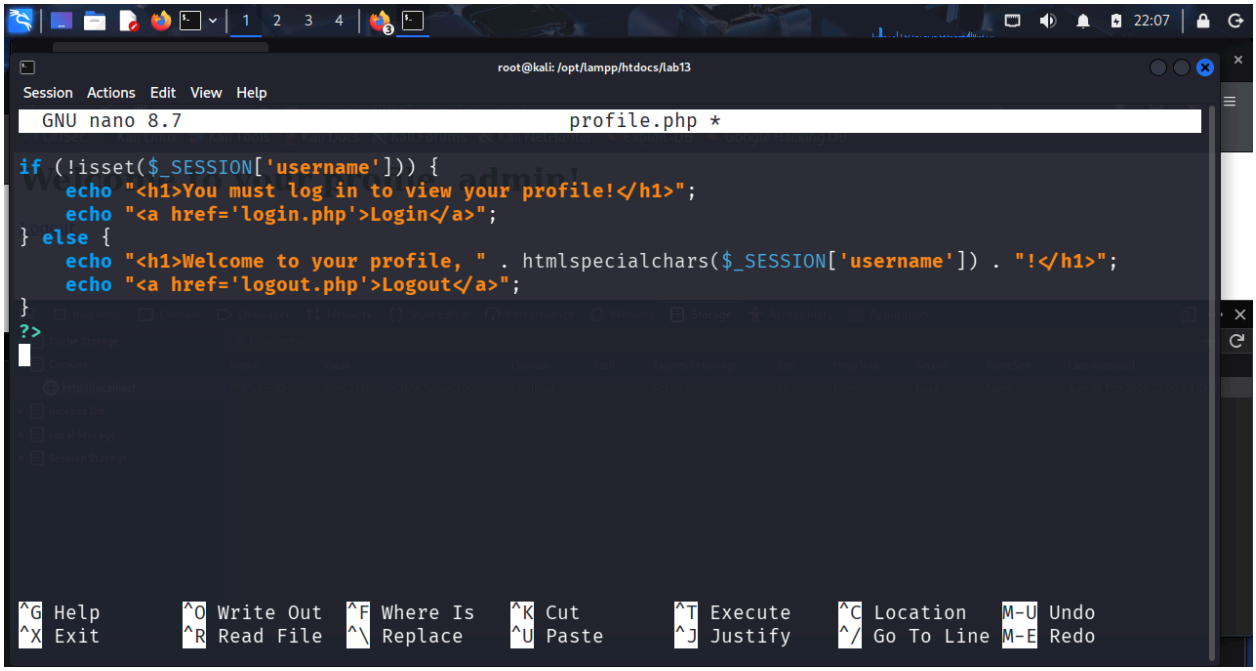
1. Secure Cookie Configuration

The session cookie was configured with:

- HttpOnly flag
- SameSite=Strict
- secure flag (depending on HTTP/HTTPS usage)

This reduces the risk of JavaScript-based cookie theft and cross-site attacks.





2. Session ID Regeneration

After successful login:

session_regenerate_id(true);

This prevents session fixation and reuse of stolen session IDs.

3. Session Timeout

A 15-minute session timeout was implemented in profile.php:

$timeout_duration = 900;

If inactive beyond this period, the session is destroyed automatically.

Task 3: Testing Mitigation

1. Retesting Session Hijacking

The hijacking process was repeated after implementing security controls.

Result:

This attack should have failed if:

- The session ID changed after login.
- The previous session ID was no longer valid.

2. Testing Session Timeout

After remaining idle beyond the timeout duration:

Result:

The system should have displayed:

Your session has expired. Please log in again.

Conclusion

This lab demonstrated how session hijacking occurs through session ID theft.
It also showed that implementing security best practices such as:

- Regenerating session IDs
- Using secure cookie flags
- Enforcing session timeouts
- Using HTTPS

significantly reduces the risk of session hijacking attacks.

The mitigation techniques successfully prevented unauthorized session reuse.